

ECE 276B Project 3: Infinite-Horizon Stochastic Optimal Control

Winston Chou
 University of California, San Diego
 PID: A17460970
 La Jolla, CA, USA
 w3chou@ucsd.edu

Abstract—This report presents the implementation and evaluation of two optimal control strategies for trajectory tracking of a unicycle car model in a stochastic environment with obstacles. We first implement a receding-horizon Certainty Equivalent Control (CEC) scheme that solves a Non-Linear Programming (NLP) problem at each time step. Second, we utilize Generalized Policy Iteration (GPI) to solve the infinite-horizon stochastic optimal control problem through value iteration on a discretized state space. We compare the performance of both methods in tracking a complex lemniscate trajectory while maintaining safety. Our results highlight the trade-offs between online optimization (CEC) and offline optimal value function V^* pre-computation (GPI) regarding tracking accuracy and real-time feasibility.

Index Terms—Trajectory tracking, stochastic optimal control, Certainty Equivalent Control, Model Predictive Control, Generalized Policy Iteration, unicycle kinematics.

I. INTRODUCTION

Autonomous robot navigation requires precise trajectory tracking capabilities alongside robust obstacle avoidance. In many practical applications, the robot's dynamics are non-linear and subject to stochastic disturbances, while the environment contains complex constraints. This project focuses on the trajectory tracking problem for a unicycle robot tasked with following a periodic lemniscate path in the presence of circular obstacles and motion noise.

We explore two foundational approaches in optimal control:

- 1) **Certainty Equivalent Control (CEC)**: A receding-horizon approach that simplifies the stochastic control problem by solving a deterministic optimization problem at each step. By ignoring noise and assuming the most likely transition, CEC leverages efficient NLP solvers to find locally optimal control sequences.
- 2) **Generalized Policy Iteration (GPI)**: A dynamic programming-based approach that addresses the stochasticity directly. By discretizing the error-state space, GPI computes an optimal value function V^* and an optimal policy π^* offline that minimizes the expected infinite-horizon discounted cost.

This report details the mathematical formulation of the unicycle dynamics and the tracking objective, describes the implementation of both algorithms, and provides a comparative analysis of their tracking performance and computational characteristics.

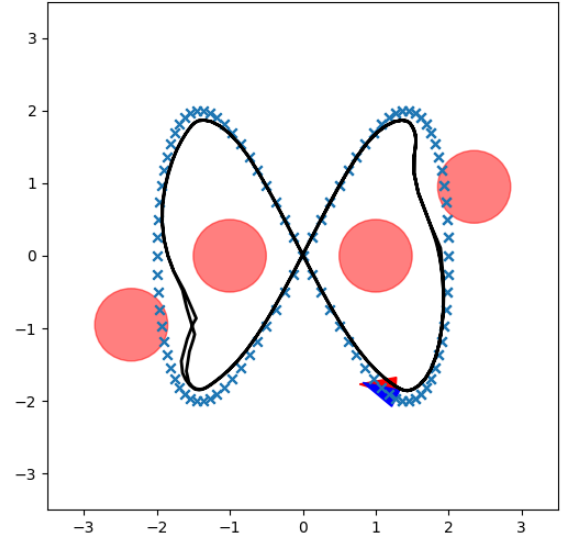


Fig. 1: Simulation of the unicycle car tracking a lemniscate trajectory in an environment with obstacles using Certainty Equivalent Control (CEC) with Mujoco Simulator.

II. PROBLEM STATEMENT

The robot is modeled as a unicycle car operating in a 2D environment $\mathcal{F} \subset \mathbb{R}^2$ with state $x_t \in \mathbb{R}^3$ and control $u_t \in \mathcal{U} \subset \mathbb{R}^2$.

A. Robot Kinematics

The continuous-time kinematics are given by:

$$\dot{x} = \begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix} \quad (1)$$

where v and ω are the linear and angular velocities. The discrete-time stochastic dynamics used in this project assume exact integration over a time step Δt :

$$x_{k+1} = f_d(x_k, u_k) + w_k, \quad w_k \sim \mathcal{N}(0, \Sigma) \quad (2)$$

where w_k represents the process noise. The nominal transition function $f_d(x_k, u_k)$ is computed by exactly integrating the

unicycle kinematics over the time step Δt , assuming constant control inputs v_k and ω_k :

$$f_d(x_k, u_k) = x_k + \begin{bmatrix} \Delta t \text{sinc}(\frac{\omega_k \Delta t}{2}) \cos(\theta_k + \frac{\omega_k \Delta t}{2}) & 0 \\ \Delta t \text{sinc}(\frac{\omega_k \Delta t}{2}) \sin(\theta_k + \frac{\omega_k \Delta t}{2}) & 0 \\ 0 & \Delta t \end{bmatrix} \begin{bmatrix} v_k \\ \omega_k \end{bmatrix} \quad (3)$$

where $\text{sinc}(x) = \sin(x)/x$ is the unnormalized sinc function.

B. Error Kinematics

To track the reference trajectory $x_{ref}(t)$, we define the tracking error as $e_k = x_k - x_{ref}(k\Delta t)$. The error kinematics describe the evolution of this error over time. Given the discrete-time dynamics $x_{k+1} = f_d(x_k, u_k) + w_k$, the error at the next time step is:

$$e_{k+1} = f_d(e_k + x_{ref,k}, u_k) - x_{ref,k+1} + w_k \quad (4)$$

where $x_{ref,k} = x_{ref}(k\Delta t)$. This formulation allows the optimal control problem to be expressed in terms of the error state, where the goal is to drive e_k to the origin.

C. Objective Function

The goal is to minimize the expected infinite-horizon discounted cost:

$$V^*(x_0) = \min_{u_0, u_1, \dots} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k \ell(e_k, u_k) \right] \quad (5)$$

where $\gamma \in (0, 1)$ is the discount factor and $\ell(e, u)$ is the stage cost:

$$\ell(e, u) = e_{pos}^T Q e_{pos} + q(1 - \cos e_\theta)^2 + u^T R u \quad (6)$$

Here $e_{pos} = [e_x, e_y]^T$, while Q , R , and q are weighting parameters.

D. Constraints

The control inputs are constrained within $[v_{min}, v_{max}]$ and $[\omega_{min}, \omega_{max}]$. Additionally, the robot must avoid a set of n circular obstacles $\mathcal{O} = \{O_j\}_{j=1}^n$ and remain within the world boundaries. Safety is enforced by ensuring the distance to any obstacle center $p_{O,j}$ satisfies:

$$\|p_x - p_{O,j}\|_2 \geq r_j + r_{robot} \quad (7)$$

where r_j is the obstacle radius and r_{robot} is the robot's collision margin.

E. System Parameters

The physical and simulation parameters used in this project are summarized in Table I. These values define the robot's capabilities, the environment's noise characteristics, and the optimization objective.

TABLE I: Robot and Simulation Parameters

Parameter	Symbol	Value
Time Step	Δt	0.5 s
Robot Radius	r_{robot}	0.3 m
Linear Velocity Limits	$[v_{min}, v_{max}]$	[0.1, 1.0] m/s
Angular Velocity Limits	$[\omega_{min}, \omega_{max}]$	[-1.0, 1.0] rad/s
Process Noise (Pos)	$\sigma_{x,y}$	0.04
Process Noise (Rot)	σ_θ	0.004
Discount Factor	γ	0.95
CEC Horizon	T_{CEC}	10
GPI Horizon	T_{GPI}	[50/0.5] = 100
State Weights	Q, q	$I_2, 1.0$
Control Weights (CEC)	R	diag(0.1, 0.1)
Control Weights (GPI)	R	diag(0.5, 0.5)
GPI Position Grid Bands	e_x, e_y	(0 $\rightarrow$ 1 : 7), (1 $\rightarrow$ 3 : 4), (3 $\rightarrow$ 6 : 3)
GPI Heading Grid Size	$ \mathcal{E}_\theta $	72
GPI Control Grid Size	$ \mathcal{V} , \mathcal{W} $	8, 15
Collision Margin (GPI)	d_{margin}	0.0
Collision Cost Weight (GPI)	λ_{coll}	10^4
Reported GPI Rollout Mode	—	Deterministic (no process noise)

III. TECHNICAL APPROACH

The trajectory tracking problem is addressed using two distinct paradigms: receding-horizon optimization and infinite-horizon value iteration.

A. Certainty Equivalent Control (CEC)

CEC simplifies the stochastic optimal control problem by assuming that process noise is zero ($w_t = 0$) during the planning horizon. At each time step k , we approximate the infinite-horizon problem by solving a finite-horizon deterministic Non-Linear Program (NLP).

Algorithm 1 Receding-Horizon CEC

- 1: Measure the current state x_k .
- 2: Formulate a finite-horizon NLP using x_k and the reference trajectory.
- 3: Solve the NLP with CasADi/IPOPT.
- 4: Apply the first control input u_k^* .
- 5: Propagate the system to obtain x_{k+1} .
- 6: Repeat at the next time step.

1) *NLP Formulation:* We define the decision variables as the sequence of controls $\mathbf{U} = [u_k^T, \dots, u_{k+T-1}^T]^T$ and error states $\mathbf{E} = [e_k^T, \dots, e_{k+T}^T]^T$. The optimization problem is:

$$\min_{\mathbf{U}, \mathbf{E}} \sum_{i=k}^{k+T-1} \gamma^{i-k} \ell(e_i, u_i) + \gamma^T \ell_{term}(e_{k+T}) \quad (8)$$

subject to:

$$\begin{aligned} e_{i+1} &= f_d(e_i + x_{ref,i}, u_i) - x_{ref,i+1}, & \forall i \in \{k, \dots, k+T-1\} \\ u_i &\in \mathcal{U}, & \forall i \in \{k, \dots, k+T-1\} \\ e_i + x_{ref,i} &\in \mathcal{F}, & \forall i \in \{k, \dots, k+T\} \end{aligned} \quad (9)$$

where $\mathcal{F}$ represents the safe set defined by the workspace boundaries $[-3, 3] \times [-3, 3]$ and the exclusion of four circular obstacles centered at (2.35, 0.95), (-2.35, -0.95), (1.0, 0.0), and (-1.0, 0.0), each with a physical radius of 0.5 m supplemented by the 0.3 m robot radius. The terminal cost

$\ell_{term}(e) = e_{pos}^T Q_{term} e_{pos} + q_{term}(1 - \cos e_\theta)^2$ is chosen to encourage tracking beyond the horizon. We utilize the CasADi framework with the *ipopt* interior-point solver to solve this NLP in a receding-horizon fashion.

2) *Optimality and Completeness*: CEC assumes its estimates of the future are certain, which makes it only *locally optimal* in the stochastic setting. Since the unicycle dynamics and obstacle constraints are non-convex, the NLP solver may converge to a local minimum rather than the global optimum. Furthermore, the approach lacks *completeness* guarantees; the solver might fail to find a feasible path even if one exists, particularly when the robot is in close proximity to obstacles or if the horizon T is insufficient to see through a narrow passage. However, in practice, the receding-horizon nature provides high-frequency feedback that compensates for moderate disturbances.

B. Generalized Policy Iteration (GPI)

GPI addresses the infinite-horizon problem by pre-computing a stationary optimal policy $\pi^*(e)$ and the optimal value function $V^*(e)$. The continuous error-state space $\mathcal{E}$ is discretized into a finite grid $\mathcal{S}_{grid}$. In the current implementation used for the reported rollouts, GPI is evaluated with deterministic transitions (process noise disabled), which simplifies the Bellman backup but does not yet capture full stochastic uncertainty. We perform two interleaved steps:

- 1) **Policy Evaluation**: Estimate the value function V^π for the current policy π .
- 2) **Policy Improvement**: Update the policy by selecting the action that minimizes the one-step lookahead cost to approach the optimal value V^* :

$$\pi'(e) = \arg \min_{u \in \mathcal{U}} \left\{ \ell(e, u) + \gamma \sum_{e'} P(e'|e, u) V^\pi(e') \right\} \quad (10)$$

Algorithm 2 Offline GPI for Periodic Tracking

- 1: Discretize the error-state space and control space.
 - 2: Initialize π and V^π on the grid.
 - 3: **repeat**
 - 4: Evaluate the current policy over the discretized state space.
 - 5: Improve the policy using one-step lookahead and the Bellman update.
 - 6: **until** the policy converges or a fixed iteration budget is reached.
 - 7: Execute the resulting policy online by table lookup.
-

1) *State and Control Discretization*: The continuous error-state space $\mathcal{E}$ is discretized using a multi-resolution multiband grid for position components (e_x, e_y) and uniform spacing for orientation error (e_θ) . This multi-resolution approach provides high resolution near the reference trajectory to preserve tracking accuracy while maintaining computational tractability across the rest of the state space. The control space $\mathcal{U}$ is similarly discretized into linear and angular velocity components.

2) *Scaling and Collision Avoidance*: To handle high dimensionality, we utilize Numba's `@njit` compilation and parallelized computation (`prange`) for pre-computing the transition matrix and conducting value function updates. The stage cost used by GPI augments tracking and control effort with an obstacle-avoidance penalty of the form

$$\ell_{GPI}(e, u) = \ell(e, u) + \lambda_{obs} \sum_{j=1}^n \exp\left(-\frac{d_j(e)^2}{\sigma_{obs}^2}\right) + \lambda_{coll} \mathbb{1}_{collision}(e) \quad (11)$$

where $d_j(e)$ is the distance from the robot to obstacle j , λ_{obs} controls the soft avoidance term, and λ_{coll} is a large collision penalty used to make unsafe grid states unattractive. This formulation encourages the robot to maintain a safe margin from obstacles without sacrificing tracking performance, but the weights must be tuned carefully because overly large collision penalties can make the policy excessively conservative while too-small penalties lead to unsafe shortcuts. Finally, periodic boundary conditions are enforced by setting $V^*(0, e) = V^*(T, e)$, allowing for the calculation of a stationary optimal policy for the periodic lemniscate trajectory.

IV. RESULTS

All simulation results were generated with a time step of $\Delta t = 0.5$ s. In the numerical simulator, which uses the exact discrete-time integration in `car_next_state()`, CEC tracks the lemniscate trajectory smoothly and remains stable around the obstacle field. This is shown in Fig. 2, where the controller follows the reference path closely while maintaining feasibility.

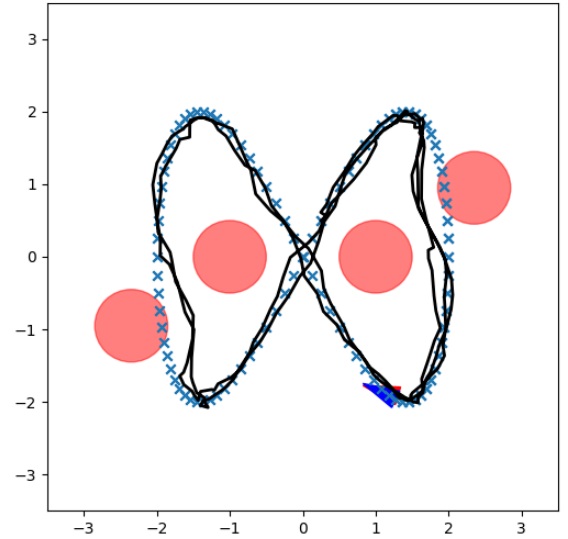


Fig. 2: CEC trajectory in the numerical simulator with exact integration and $\Delta t = 0.5$ s.

We also tested the same controller in Google DeepMind’s MuJoCo simulator. The resulting motion appears smoother, as shown in Fig. 3, which is expected because the MuJoCo model captures vehicle geometry and continuous contact effects more faithfully than the simplified kinematic rollout. Although the MuJoCo vehicle is implemented as a car-like Ackermann drive rather than an ideal unicycle, the two models share the same nonholonomic constraint structure, so the tracking behavior is still directly comparable at the level of planar motion and heading evolution.

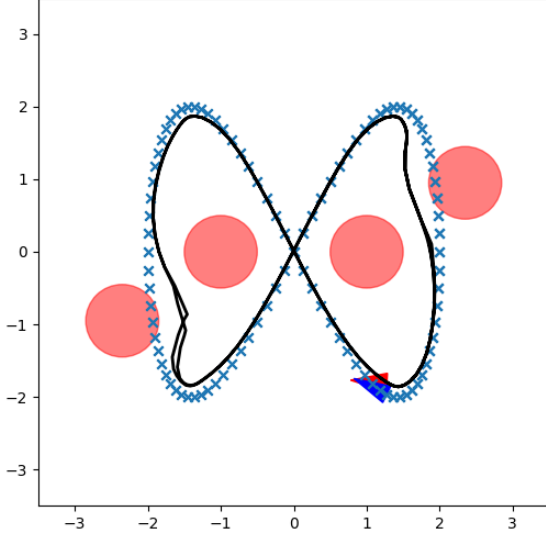


Fig. 3: CEC trajectory in Google DeepMind’s MuJoCo simulator. The motion looks smoother than the purely numerical rollout.

Compared with CEC, the GPI implementation is still unfinished and should be viewed as a preliminary baseline rather than a fully tuned controller. Offline, it already reflects the structure of the infinite-horizon problem through value iteration on a discretized state space, but the current rollout is generated under a deterministic model (no process noise), and performance is still limited by grid resolution, transition approximation, and remaining implementation work needed to make it as mature as the CEC pipeline. Fig. 4 shows the current GPI rollout; it captures the intended lemniscate shape but is less consistent near sharp turns and obstacle-adjacent regions.

The main implementation difficulty is the discretization itself. The state space must be fine enough to resolve the periodic lemniscate tracking task, especially near the self-intersection and the obstacle boundaries, but a fine grid quickly makes the transition tensor and policy-evaluation loop expensive. The control space has the same issue: coarse action grids make the policy brittle, while denser grids increase the number of lookups in every improvement step and amplify the offline

computation cost. On top of that, the loss weights are difficult to tune over a long periodic horizon because the controller must balance tracking, smoothness, and safety at every point along the cycle. If the tracking terms dominate, the policy cuts too aggressively through narrow regions; if the collision-avoidance terms dominate, the policy becomes conservative and may drift away from the reference. In practice, the best GPI behavior comes from a compromise between discretization resolution and penalty weights, and that compromise is much harder to tune than the receding-horizon CEC setup.

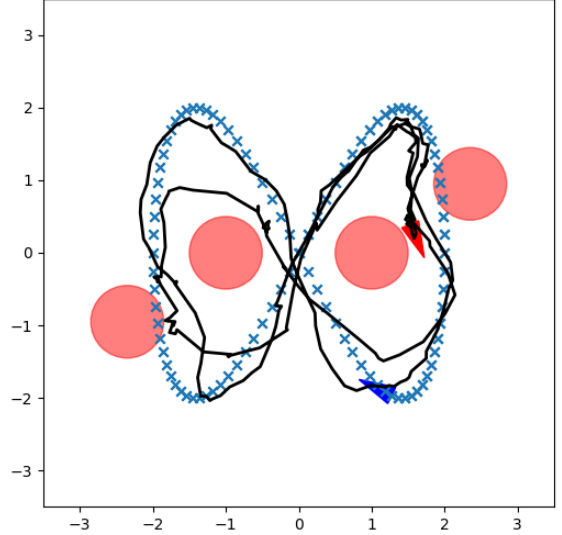


Fig. 4: Preliminary GPI rollout at $\Delta t = 0.5$ s. The implementation is not yet as complete or as well tuned as CEC.

To make the comparison explicit, Table II reports the measured runtime and cumulative tracking errors from our runs at $\Delta t = 0.5$ s.

TABLE II: Computation and Tracking Results ($\Delta t = 0.5$ s)

Method	Avg ms	E_{trans}	E_{rot}
CEC (Numerical)	69.22	55.14	43.74
CEC (MuJoCo)	692.83	65.94	28.99
GPI (No-noise)	0.37	142.52	94.99

These results show the current trade-off in this project stage: deterministic GPI provides very fast online control lookup but significantly larger tracking errors in its present form, while CEC yields better tracking in both simulators at higher per-step compute cost. MuJoCo-CEC is slower due to simulator overhead and richer dynamics, but still maintains competitive tracking behavior. A complete infinite-horizon stochastic solution should explicitly incorporate a stochastic transition model in the GPI backups; however, due to project time constraints, this full stochastic extension was not implemented in the current work.

Overall, the experiments suggest that CEC is the more complete and reliable controller in the current project state: it is already working well in both the exact numerical simulator and MuJoCo, while GPI remains an incomplete offline optimal-control baseline whose final performance will depend on further refinement of the discretization and policy-iteration loop.

V. CONCLUSION

In this project, we compared receding-horizon optimization (CEC) and value iteration (GPI) for unicycle trajectory tracking. CEC is the more mature solution in our implementation: it works well at $\Delta t = 0.5$ s, tracks the reference accurately in the exact numerical simulator, and also transfers cleanly to MuJoCo. GPI remains a useful conceptual benchmark for infinite-horizon stochastic control, but the current version is both unfinished and evaluated in a deterministic setting (no process noise), so additional development is needed before making a full stochastic comparison with CEC. The main trade-off is therefore between the online computational cost of CEC and the offline development effort needed to complete a strong GPI implementation.

REFERENCES

- [1] N. Atanasov, “ECE276B: Planning & Learning in Robotics – Schedule,” University of California San Diego.
- [2] D. P. Bertsekas, “Dynamic Programming and Optimal Control,” Athena Scientific.